



Universidad de
La Sabana

ECOMMIFY

Diseño Conceptual y Lógico de la Base de Datos

Arquitectura híbrida polígloa: PostgreSQL + MongoDB

Equipo de trabajo

Jorge Eliecer Rojas Quiñones - 0000393134

David Panesso Sánchez - 0000399159

Fabian Andrés Rojas García - 0000393714

Juan Esteban Gómez Roa - 0000393177

Presentado a:

Miguel Alfonso Varela Fonseca

Estructura de la presentación

01 Contexto y análisis de requisitos

02 Modelo ER normalizado

03 Justificación técnica

04 EDA del dataset Olist

05 Matriz de decisiones + CAP

06 Conclusiones y cierre

Ecommify sobre el dataset Brazilian E-Commerce de Olist

9

archivos CSV

~1.45M

registros agregados

7

tablas PostgreSQL

2

colecciones MongoDB

Arquitectura híbrida políglota

PostgreSQL — 7 tablas

Núcleo OLTP · ACID · CP

5 originales: customers · sellers · product_category_translation · order_items · order_payments

2 transformadas: products_v2 · orders_v2

MongoDB — 2 colecciones

Semi-estructurado · BASE · AP

reviews — texto libre + sentiment
geolocation — datos geoespaciales 2dsphere

Referencias lógicas (sin FK física) con SQL

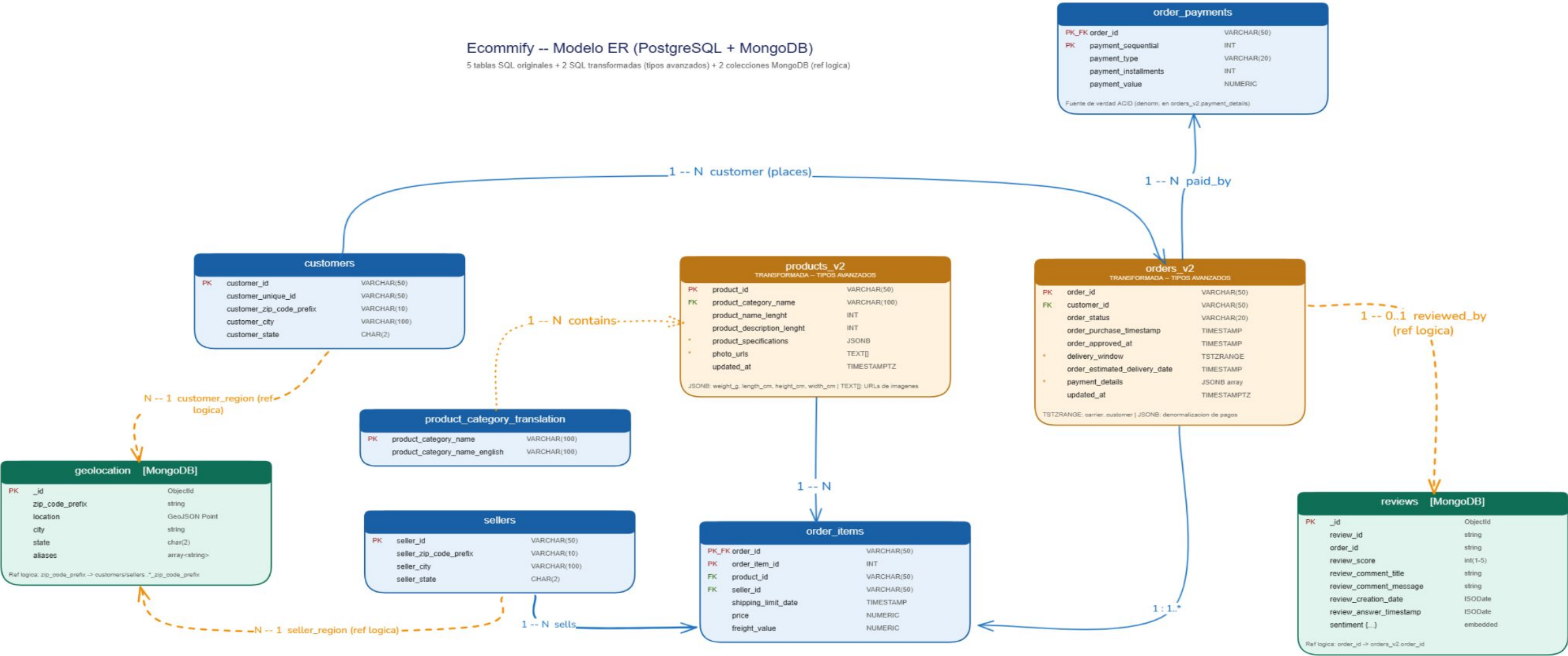
Requisitos funcionales y no funcionales

Funcionales	
RF01	Trazabilidad de estado del pedido (created → approved → shipped → delivered → canceled)
RF02	Múltiples métodos de pago con cuotas (credit_card, boleto, voucher, debit_card)
RF03	Catálogo con atributos físicos variables en JSONB y URLs reales en TEXT[]
RF04	Reseñas de texto libre con score 1-5 en colección reviews (MongoDB)
RF06	Búsqueda fuzzy de productos con pg_trgm
RF07	Reportes analíticos diarios vía vistas materializadas

No funcionales	
RNF01	Consultas transaccionales < 400 ms
RNF02	Disponibilidad ≥ 99.5%
RNF03	Consultas analíticas máx. 5 segundos
RNF04	Consistencia 100% dentro de cada motor
RNF05	Free tier: 512 MB MongoDB Atlas M0 · 500 MB Supabase
RNF06	Búsqueda fuzzy < 1 segundo

Diagrama ER híbrido: 5 originales + 2 transformadas + 2 Mongo

Ecommify -- Modelo ER (PostgreSQL + MongoDB)
5 tablas SQL originales + 2 SQL transformadas (tipos avanzados) + 2 colecciones MongoDB (ref logica)



Las relaciones cross-database (naranjas punteadas) son logicas: no existen como FK fisicas. Se sincronizan a nivel aplicativo (eventos asincronos / CDC).
products_v2 y orders_v2 son las versiones transformadas con tipos avanzados (JSONB, TEXT[], TSTZRANGE). Reemplazan a las tablas products y orders del esquema original.

9 entidades del dominio · FK físicas + ref. lógicas

PostgreSQL — 7 tablas (FK física)	
customers	PK customer_id · ~99k
sellers	PK seller_id · ~3k
product_category_translation	PK product_category_name · 71
order_items	PK (order_id, order_item_id) · ~113k
order_payments	PK (order_id, payment_seq) · ~104k
products_v2	Transformada · JSONB + TEXT[] · ~33k
orders_v2	Transformada · TSTZRANGE + JSONB · ~99k

MongoDB — 2 colecciones (ref. lógica)	
reviews	Texto libre + sentiment · ~99k docs
geolocation	GeoJSON Point + aliases · ~1M docs

Relaciones lógicas sin FK física

ORDERS_V2 ↔ reviews · 1 : 0..1

CUSTOMERS ↔ geolocation · N : 1

SELLERS ↔ geolocation · N : 1

Sincronización vía aplicación + eventos asíncronos

Trade-off aceptado a cambio de las ventajas del modelo políglota

Decisiones que materializan la 3FN

1FN

Atomicidad con excepciones intencionales

Tablas originales cumplen 1FN. Excepción: `product_specifications` (JSONB), `photo_urls` (TEXT[]) y `delivery_window` (TSTZRANGE). Se leen como bloques atómicos, no se filtran subvalores con joins.

2FN

Sin dependencias parciales en PKs compuestas

En `order_items` (PK = `order_id` + `order_item_id`) y `order_payments` (PK = `order_id` + `payment_sequential`): cada atributo no clave depende de la combinación completa de la clave.

3FN

Sin dependencias transitivas

- (a) `product_category_translation` extraída como tabla de referencia (71 filas).
- (b) Información geográfica delegada a `geolocation` (MongoDB), eliminando `zip`→`city`→`state` en `customers` y `sellers`.

ACID vs BASE — paradigmas complementarios

ACID — PostgreSQL

Atomicidad

Orden + ítems + pagos como una transacción única. Rollback completo si falla.

Consistencia

FK físicas y CHECK aseguran integridad de dominio. `payment_value > 0` a nivel motor.

Aislamiento

Lecturas y escrituras concurrentes no se bloquean.

Durabilidad

Una vez COMMIT, el dato persiste aunque caiga el servidor.

BASE — MongoDB

Basically Available

Las colecciones siempre responden a una lectura, aunque la réplica no tenga el dato más reciente.

Soft state

El estado puede cambiar sin input externo (TTL, expiración de cachés).

Eventual consistency

Las réplicas convergen al mismo estado en tiempo corto.

Aplicación a Ecommify

Reviews insertadas post-entrega (asíncrono) · geolocation read-mostly.

Transformación de products y orders → products_v2 y orders_v2

Tabla origen	Campos origen	Tabla destino	Campo destino	Tipo	Índice
products	weight_g, length_cm, height_cm, width_cm	products_v2	product_specifications	JSONB	GIN
products	URLs reales de imágenes (NO photos_qty)	products_v2	photo_urls	TEXT[]	GIN
orders	delivered_carrier_date, delivered_customer_date	orders_v2	delivery_window	TSTZRANGE	GIST

Justificación rápida

product_specifications (JSONB)

Agrupar 4 atributos físicos consultados en bloque. Permite extender (color, RAM, etc.) sin ALTER TABLE. Índice GIN para queries por contenido.

photo_urls (TEXT[])

El dataset Olist sólo guardaba product_photos_qty (cantidad). Ecommify almacena las URLs reales en array nativo; cardinality(photo_urls) reemplaza al INT redundante.

delivery_window (TSTZRANGE)

Combina carrier_date + customer_date en un rango temporal. Operadores nativos @>, &&. Índice GIST para análisis de SLA por periodo.

Extensiones replanteadas + estrategia de partición

pg_trgm

ACTIVADA

Búsqueda fuzzy en nombres de producto y categorías.
Operadores % y similarity().

*Ej.: 'celar' encuentra 'celular'.
Índice GIN con gin_trgm_ops sobre product_category_name.*

PostGIS

DESCARTADA

Originalmente prevista para fletes basados en distancia.

*Al delegar geolocation a MongoDB, las coordenadas e índice 2dsphere residen ahí.
PostGIS no tendría datos sobre los cuales operar.*

Estrategia de particionamiento — orders_v2

PARTITION BY RANGE (order_purchase_timestamp)

Justificación: alto volumen acumulado, consultas con filtro temporal frecuentes, necesidad de separar datos calientes vs fríos.

orders_2017

Partición por año natural

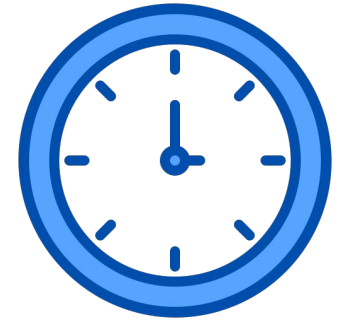
orders_2018

Partición por año natural

orders_default

Fechas fuera de rango definido

Mantenimiento programado con pg_cron



`mv_sales_by_category_monthly`

pg_cron permite agendar jobs SQL directamente dentro de PostgreSQL. En Supabase se habilita desde Database > Extensions. Los jobs se ejecutan diariamente en horario de baja carga.

El refresh diario mantiene las vistas frescas para los dashboards sin afectar el rendimiento OLTP (se ejecuta a las 3 AM). El VACUUM ANALYZE diario libera tuplas muertas y actualiza estadísticas del planner para que las consultas con tipos avanzados (GIN/GIST) sigan eligiendo los índices correctos.

Refresh diario · 03:00 AM

EDA del dataset Olist



Siete de los nueve datasets presentan integridad referencial perfecta. Los nulos en orders y reviews son nulos válidos estructurales que no representan errores sino estados legítimos del negocio. Saber hacer esta distinción es fundamental en modelado de datos: no todo valor faltante es un problema que hay que resolver.

El caso más interesante desde el punto de vista de calidad es products, con 610 registros sin categoría. Estos sí son datos faltantes reales que requieren una estrategia de imputación antes de cargarlos en producción, ya que la categoría es un atributo clave para la mayoría de las consultas analíticas.

- **Inventario del dataset (9 archivos · ~1.45M registros)**
 - **Calidad de datos (nulos, duplicados, integridad)**
- **Distribuciones, análisis bivariado, patrones temporales y geográficos**
 - **Outliers · Cómo el EDA llevó al diseño**

Motor por entidad — criterios objetivos

Entidad	Motor	Volumen	Estructura	Consistencia
customers	PostgreSQL	~99k	Rígida y estable	Fuerte (ACID)
sellers	PostgreSQL	~3k	Rígida	Fuerte
product_category_translation	PostgreSQL	71	Catálogo	Fuerte
order_items	PostgreSQL	~113k	Rígida	Fuerte
order_payments	PostgreSQL	~104k	Rígida	Crítica (financiera)
products_v2	PostgreSQL	~33k	Rígida + JSONB / TEXT[]	Fuerte
orders_v2	PostgreSQL	~99k	Rígida + TSTZRANGE / JSONB	Fuerte
reviews	MongoDB	~99k	Variable + sentiment	Eventual
geolocation	MongoDB	~1M	Variable + aliases	Eventual

Criterios: volumen · estructura · patrones de consulta · requisitos de consistencia.

CP en el núcleo OLTP · AP en el módulo semi-estructurado

Solo dos de tres en presencia de particiones de red — la elección se hace por motor.

PostgreSQL

CP

Consistencia + Tolerancia a particiones

Durante un fallo controla un error antes de registrar una orden inconsistente.

Aplicación a Ecommify:

Pedidos, pagos y catálogo. Integridad referencial no negociable en estos módulos.

MongoDB

AP

Disponibilidad + Tolerancia a particiones

Una review insertada en un nodo aislado se replicará al restablecer la conexión, sin bloquear la experiencia del usuario.

Aplicación a Ecommify:

Reviews y geolocation. La consistencia eventual es suficiente porque no condicionan transacciones financieras.

Decisiones costo controlado

1

Arquitectura polígloa vs complejidad operacional

Trade-off

Dos motores → dos conexiones, dos monitoreos, dos backups, dos curvas de aprendizaje.

Mitigación

Cada motor resuelve un problema que el otro no puede resolver eficientemente. Proveedores gestionados (Supabase + Atlas) absorben la mayor parte de la carga operacional.

2

Desnormalización en orders_v2 (JSONB de pagos)

Trade-off

payment_details rompe 3FN. Duplicidad de verdad entre order_payments (ACID) y el JSONB embebido.

Mitigación

order_payments designada como fuente de verdad; el JSONB es solo caché de lectura para evitar JOINs.

3

Pérdida de 1FN con tipos avanzados

Trade-off

JSONB / TEXT[] / TSTZRANGE rompen 1FN. Más difícil filtrar subvalores con SQL estándar.

Mitigación

Aceptado intencionalmente: los patrones de consulta leen estos campos como bloques atómicos. Operadores @>, &&, ANY() son la única interfaz documentada.



Universidad de
La Sabana

Gracias

Equipo Ecommify

Jorge E. Rojas · David Panesso · Fabian A. Rojas · Juan E. Gómez